Paralelização do Jogo da Vida com OpenMP: implementação e avaliação experimental

Victor Hugo Fernandes Martins e Vinicius da Silva Montibeller

Universidade Federal de Santa Catarina (UFSC) – Departamento de Informática e Estatística

victorhugofernandesmartins@gmail.com e vinism96@hotmail.com

Resumo. Este trabalho compara duas estratégias de paralelização do Jogo da Vida em C com OpenMP e uma implementação sequencial. A versão simple paraleliza as linhas da grade a cada geração, enquanto a versão optimized mantém uma região paralela ao longo da simulação. A distribuição estática foi escolhida pela regularidade do custo de atualização das células e pela localidade dos dados na representação unidimensional da matriz. Os resultados mostram que o benefício depende do tamanho da entrada: nas menores, o overhead limita os ganhos, nas maiores, o speedup chega a aproximadamente 10,60 vezes com 16 threads. A eficiência diminui ao utilizar 32 threads, e a versão otimized não supera constantemente a simple

Abstract. This paper compares two OpenMP parallelization strategies for the Game of Life in C with a sequential implementation. The simple version parallelizes grid rows at each generation, whereas the optimized version keeps a parallel region active throughout the simulation. Static scheduling was chosen because cell updates have relatively uniform computational cost and the one-dimensional grid representation favors data locality. Results show that benefits depend on input size: overhead limits gains for smaller inputs, while the largest input reaches a speedup of approximately 10.46 with 16 threads. Efficiency decreases with 32 threads, and the optimized version does not outperform the simple version in every configuration.

Palavras-chave: Jogo da Vida, programação paralela, OpenMP, C, desempenho.

1. Introdução

Autômatos celulares são modelos computacionais constituídos por uma grade de células que são modificados em etapas de acordo com regras selecionadas. Jogo da Vida de Conway é um exemplo desse modelo, no qual o estado de cada célula na geração seguinte depende de seu estado atual e do número de vizinhos vivos. Neste trabalho, considera-se uma variação do problema em que cada posição da grade bidimensional, de dimensões (L \times C), possui uma regra de evolução própria, expressa na notação B/S, que permanece fixa durante a simulação. Cada célula considera até oito vizinhos, respeitando os limites da grade e sem utilizar bordas toroidais.

No Jogo da Vida, o estado de cada célula na geração seguinte depende de seu estado atual, do número de vizinhos vivos e da regra associada à posição, utilizando diferentes regras B/S sem bordas toroidais. Como cada célula pode ser calculada a partir

da geração corrente, existe paralelismo de dados entre as atualizações, desde que a grade corrente permaneça inalterada durante o cálculo.

O objetivo deste trabalho é comparar duas implementações paralelas em C com OpenMP, denominadas simple e optimized, com uma implementação sequencial. A análise enfatiza como o tamanho do problema influencia o speedup e a eficiência quando se aumenta o número de threads.

2. Estratégia de paralelização

A versão simple utiliza a diretiva #pragma omp parallel for schedule(static) no laço externo que percorre as linhas da grade. Essa estratégia é adequada à regularidade do Jogo da Vida: cada célula realiza operações semelhantes para determinar seu estado na geração seguinte. A distribuição estática divide previamente as iterações entre as threads, reduzindo o custo de escalonamento. Visto que a matriz é representada por um vetor unidimensional (por exemplo, pos = row * cols + col), as células de uma mesma linha ocupam posições contíguas na memória. A divisão por linhas favorece a localidade espacial e reduz a necessidade de redistribuir o trabalho durante a execução.

Na versão optimized, a região paralela envolve o laço de gerações, evitando criar uma nova região paralela a cada geração. Dentro dela, #pragma omp for distribui o cálculo das linhas entre as threads. A barreira implícita ao final do omp for garante que todas as células da geração seguinte estejam prontas antes da troca dos buffers. Em seguida, #pragma omp single realiza a troca dos ponteiros current e next. Nas duas versões, current e next são buffers separados: cada thread lê exclusivamente os valores da geração atual e escreve em posições distintas da próxima geração. Essa independência dentro de uma geração preserva a evolução síncrona e evita copiar a grade inteira a cada iteração. A combinação de distribuição por linhas e buffers separados a implementação relativamente simples e acertiva.

Outras estratégias foram cogitadas durante a execução do projeto, mas é importante distinguir alternativas analisadas teoricamente de estratégias avaliadas experimentalmente. O collapse(2) combina os espaços de iteração dos dois laços, schedule(dynamic) distribui novas iterações conforme as threads terminam, schedule(guided) distribui blocos progressivamente menores, task/taskloop divide o processamento em tarefas gerenciadas pelo runtime, blocos bidimensionais dividem a matriz em regiões menores. Como o trabalho por célula é relativamente uniforme e há linhas suficientes para distribuir entre as threads, essas alternativas não apresentam uma vantagem evidente e podem aumentar o gerenciamento ou a complexidade sem levar a ganho de eficiência e velocidade.

Tabela 1. Estratégias de paralelização também consideradas

Estratégia	Conceito OpenMP	Justificativa para não priorizar
collapse(2)	Combina os espaços de iteração dos dois laços.	A divisão por linhas já oferece iterações suficientes, o ganho adicional não é garantido.
schedule(dynamic)	Distribui novas iterações conforme as threads terminam.	O trabalho por linha é aproximadamente uniforme, o escalonamento dinâmico pode adicionar overhead.
schedule(guided)	Distribui blocos progressivamente menores.	É mais útil quando há desbalanceamento significativo, que não foi demonstrado.
task / taskloop	Divide o processamento em tarefas gerenciadas pelo runtime.	Acrescenta gerenciamento de tarefas a um trabalho regular que pode ser distribuído com omp for.
Blocos bidimensionais	Divide a matriz em regiões menores.	Pode melhorar a localidade de cache, mas aumenta a complexidade

3. Implementação e ambiente experimental

As implementações foram desenvolvidas em C e executadas no cluster Santos Dumont, na partição sequana_cpu_dev. O script de submissão ao SLURM solicita um nó, uma tarefa e 32 CPUs por tarefa. A configuração do ambiente OpenMP desabilita o ajuste dinâmico do número de threads e define a vinculação das threads aos núcleos disponíveis por meio das variáveis OMP_DYNAMIC=FALSE, OMP_PROC_BIND=TRUE e OMP_PLACES=cores.

A compilação foi configurada por meio de um Makefile, utilizando GCC com as opções -O3 -Wall -Wextra para a versão sequencial e as mesmas opções acrescidas de -fopenmp para as versões paralelas. A versão do GCC identificada no terminal foi a 8.5.0. O processador observado no ambiente foi um Intel Xeon Gold 6152, com arquitetura x86_64, dois sockets, 22 núcleos por socket e dois threads de hardware por núcleo.

Os experimentos utilizaram sete entradas, de life-1.in a life-7.in, com dimensões que variam de 5 × 5 a 480 × 640 células e números de gerações entre 3 e 400. As grades possuem entre 25 e 307.200 células. Essa variação permite avaliar o desempenho das implementações em diferentes escalas de processamento.

Foram avaliadas três versões do programa: a sequencial, denominada seq, e as paralelas simple e optimized. Os testes paralelos utilizaram 2, 4, 8, 16 e 32 threads, com dez repetições por configuração. O tempo de execução foi medido utilizando o comando interno time do Bash, configurado com TIMEFORMAT='%6R', que registra o tempo real decorrido em segundos com resolução de milissegundos. A medição abrange a execução completa de cada programa, incluindo a leitura da entrada, o processamento e a finalização.

A validação foi realizada antes das medições. Para cada entrada, a saída da versão sequencial foi utilizada como referência para comparação com as saídas das duas versões paralelas, por meio do comando cmp. Quando havia um arquivo de saída esperado correspondente à entrada, ele também era comparado com a referência sequencial. Dessa forma, o script verifica a igualdade dos arquivos de saída antes de iniciar as repetições utilizadas no benchmark.

4. Resultados e análise

Os resultados evidenciam que o tamanho da entrada influencia fortemente o benefício da paralelização. Nas quatro entradas menores, os tempos sequenciais medianos variam de aproximadamente 0,002 a 0,003 segundo, e as medições com até 16 threads permanecem praticamente iguais às sequenciais. Nesses casos, a eficiência trazida pela paralização é muito pequena devido ao custo de coordenação e criação das threads, de modo que o speedup observado fica próximo de 1, ademais nesses casos pode se analisar que o aumento de threads reduz a eficiência. Tanto que com 32 threads, os tempos medianos pioram em relação à execução sequencial em várias dessas entradas.

Nas entradas maiores, a redução do tempo é mais expressiva. Em tabelas maiores, o maior speedup informado chega a 8,2581× para optimized com 32 threads, a versão simple atinge 8,0× com 16 threads. Na maior tabela testada (life-7.in) a versão simple alcança speedup de 10,5987× com 16 threads, enquanto a optimized chega a 10,1155× com 16 threads. Esses resultados mostram que uma carga maior permite amortizar o overhead e aproveitar melhor o paralelismo.

O ganho não cresce indefinidamente com o número de threads. Em life-7.in, passar de 16 para 32 threads reduz o speedup de 10,5987× para 8,4682× na versão

simple e de 10,1155× para 9,4815× na optimized. A eficiência cai para 26,46% na simple e 29,63% na optimized com 32 threads.

Tabela 2. Speedup e eficiência em life-7.in, calculados a partir das medianas informadas no CSV.

Threads	Speedup simple	Eficiência simple	Speedup optimized	Eficiência optimized
2	2,0036×	100,18%	1,9952×	99,76%
4	3,5940×	89,85%	3,5823×	89,56%
8	6,4247×	80,31%	6,4247×	80,31%
16	10,5987×	66,24%	10,1155×	63,22%
32	8,4682×	26,46%	9,4815×	29,63%

4.1. Procedimento para os cálculos

Foram realizadas dez amostras para cada configuração de entrada, versão e número de threads. Para comparar o desempenho, utilizou-se o speedup informado no CSV, calculado pela razão entre o tempo mediano da execução sequencial e o tempo mediano da execução paralela:

$$S(p) = \frac{T_{seq}}{T_{par(P)}}$$

em que $S(p)$ representa o speedup com p threads, T_{seq} é o tempo mediano da execução sequencial e $T_{par(p)}$ é o tempo mediano da execução paralela com p threads. A eficiência é calculada pela razão entre o speedup e o número de threads, expressa em porcentagem:

$$E(p) = \frac{S(p)}{p} * 100\%$$

Os valores de speedup e eficiência apresentados na análise foram mantidos conforme o CSV fornecido. Para a entrada life-7.in, por exemplo, o tempo mediano sequencial foi de 1,664 s. Com 16 threads, a versão simple apresentou speedup de 10,5987× e eficiência de 66,24%, enquanto a versão optimized obteve speedup de 10,1155× e eficiência de 63,22%. Com 32 threads, esses valores passaram para 8,4682× e 26,46% na versão simple, e 9,4815× e 29,63% na versão optimized.

Nas quatro entradas menores, cujos tempos medianos sequenciais variam de aproximadamente 0,002 a 0,003 s, a curta duração das execuções dificulta observar ganhos expressivos e torna os resultados mais sensíveis ao custo de gerenciamento das threads e às variações de medição.

5. Discussão

A versão simple cria e encerra uma região paralela a cada geração, enquanto a optimized mantém a equipe de threads ao longo da simulação. Apesar da vantagem teórica de reduzir o custo repetido de gerenciamento, os resultados mostram desempenho semelhante em vários casos. Na maior entrada, simple apresenta speedup ligeiramente superior com 16 threads, em life-6.in, optimized apresenta seu maior speedup com 32 threads, enquanto simple é melhor com 16. Portanto, os dados não sustentam que uma versão seja universalmente superior.

A escalabilidade depende da carga de trabalho. Para as entradas pequenas, aumentar o número de threads não reduz de forma relevante o tempo total e pode piorá-lo, especialmente com 32 threads. Na maior entrada, o speedup cresce até 16 threads e cai ao usar 32. Esse padrão é compatível com overhead de coordenação e limites de recursos compartilhados, embora os tempos, por si só, não permitam atribuir a queda a uma única causa.

6. Conclusão

O trabalho compara duas estratégias OpenMP para o Jogo da Vida: a versão simple, que paraleliza o laço de linhas com parallel for e escalonamento estático a cada geração, e a versão optimized, planejada para manter uma região paralela ativa durante a evolução. Ambas usam buffers separados para preservar a leitura da geração corrente e a escrita da próxima geração.

Conclui-se que a distribuição por linhas é uma estratégia simples e adequada à regularidade do Jogo da Vida. O benefício da paralelização cresce com o tamanho da entrada: nas menores, o overhead limita os ganhos, na maior, o speedup chega a 10,5987× com 16 threads na versão simple. Entretanto, a eficiência diminui ao usar 32 threads, e a versão optimized não supera a simple em todas as configurações. Assim, a escolha do número de threads deve considerar o tamanho da grade e os resultados medidos.

Declaração de uso de I.A

Durante o desenvolvimento deste trabalho, foram utilizadas ferramentas de inteligência artificial como apoio à elaboração de scripts auxiliares para execução e à revisão e organização textual do relatório. As sugestões fornecidas foram avaliadas pelos autores, que permaneceram responsáveis pela interpretação dos resultados e conteúdo final apresentado.

Referências

[1] Universidade Federal de Santa Catarina. INE 5645 – Programação Paralela e Distribuída. Trabalho 1: Simulação paralela de autômatos celulares. Semestre 2026/2.

[2] OpenMP Architecture Review Board. OpenMP Application Programming Interface Specification. Disponível em: https://www.openmp.org/specifications/.

[3] WIKIPEDIA CONTRIBUTORS. *Conway's Game of Life*. Wikipedia, The Free Encyclopedia. Disponível em: https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life.